

Sandboxing

Building agents by building their harness

Lesson 7 of 10 · Production

Where we left off

Lesson 6 added guards.

A gate before each tool. Limits on the run.

What a guard can't do

A guard decides **whether** a command runs.

Once it runs, it has the whole machine.

An allowed command

The person said yes.

Or it was on the safe list.

Now it can read any file, use the network, and run as long as it likes.

Guards read text

A guard looks at the command **as text**.

Text can be disguised.

A shell command can do more than it looks like.

So change the question

Not: "is this command safe?"

But: **"what can any command reach?"**

Sandboxing

Run the tools where they can reach only what you allow.

Where does it live?

Sandboxing is **built on output**.

Why output

Output is how model outputs get **run as tools**.

Lesson 2: output decides **where tools run**.

This machine, a container, a remote host.

Going further, Lesson 2

"Where tools run" was already on the list.

Today we pick one answer.

What changes

Only **where** the command executes.

Not what the model sees.

Not when the loop stops.

The enforcement

Not a text check.

The **kernel** enforces the limits.

The command can't talk its way past it.

The tool

A **container**, with Docker.

The worked example

`production/07-sandboxing/quark.py`

Lesson 6's `quark.py` plus a sandbox.

94 lines.

Two things change

1. a `sandbox( )` starts a container once
2. tool commands run **inside** it

New constants

- `IMAGE : python:3.13-slim`
- `TIMEOUT : 30 seconds`

New import

```
atexit .
```

It cleans the container up when quark exits.

sandbox()

Starts one container.

Called once, **before the loop**.

One container per run

Named `quark-<run id>` .

Same run id as the trace.

Easy to find. Easy to match.

It runs in the background

```
docker run -d , then sleep infinity .
```

It waits for commands.

Removed when done

`--rm`, plus `atexit` running `docker rm -f`.

The container goes when the run ends.

What it wrote to the mounted folder stays, and a `kill -9` skips `atexit`.

The limits, one by one

Each is a flag on `docker run`.

Limit: no network

```
--network none
```

The command cannot reach the internet.

Or anything else on the network.

Limit: memory

```
--memory 512m
```

Use more? The kernel kills it.

Limit: CPU

```
--cpus 1
```

One core. No more.

Limit: processes

```
--pids-limit 128
```

A runaway loop of processes hits a wall.

Limit: privileges

```
--cap-drop ALL
```

```
no-new-privileges
```

It cannot gain powers it wasn't given.

Limit: filesystem

```
--read-only
```

The container's own files cannot change.

One writable place

```
--tmpfs /tmp
```

```
HOME=/tmp
```

Scratch space that vanishes with the container.

Run as you

```
--user uid:gid
```

Files it creates belong to you, not to root.

The project folder

`-v cwd:cwd` and `-w cwd`

Your working directory is mounted at the same path.

The agent can do its job on your files.

Why the same path

The system prompt says where it is.

`os.getcwd()` is true inside the box too.

No confusion about paths.

What it can still reach

The project folder, writable.

Nothing else on your machine.

Not your environment either: the API key stays with the harness, outside the box.

If Docker fails

quark **exits**.

There is no fallback to running on the host.

Why no fallback

A sandbox that quietly turns off is worse than none.

You'd believe you were safe.

Running a command

Before: `subprocess.run(cmd, shell=True)` .

Now: `docker exec` .

The exec line

```
docker exec box timeout -s KILL 30 sh -c cmd
```

The command runs inside the container, under a **time limit**.

A time limit

```
timeout -s KILL 30
```

Past 30 seconds, it is killed.

Lesson 2 listed this: failure and hang.

Exit 137

A killed process exits with **137**.

quark spots it.

Telling the model

On 137, the result gains:

```
(killed: ran over 30 seconds or out of memory)
```

Why say so

The model can't see the kernel.

It only sees the result.

So the harness names the cause.

What didn't change

The model asks for a bash command.

Output runs it.

The result goes back.

Same loop.

Guards still there

The gate from Lesson 6 is still before the tool.

Sandboxing doesn't replace it.

Two layers.

Run it

Start in a scratch folder: the box mounts it.

```
mkdir /tmp/demo && cd /tmp/demo
uv run --project /path/to/building-agents \
  /path/to/building-agents/production/07-sandboxing/quark.py \
  "write hello.txt containing hi, then try /etc, the network and your env"
```

You need Docker running.

Starting Docker

If Docker isn't running, quark exits.

Start the daemon first.

Pull `python:3.13-slim` once.

See the box

While it runs:

```
docker ps
```

You'll see `quark-<run id>` .

Proof of no network

Ask it to fetch a web page.

It fails. The container has no network.

So the sandbox is the second line

Don't count on the model refusing.

The kernel limit is there either way.

Going further

`quark.py` is one way to do it.

`sandboxing.py` is a richer one.

sandboxing.py

Built on Lesson 3's `control_flow.py` .

A different sandbox shape.

Copy the project in

Not mounted. **Copied.**

Via `tar`, into the container.

What stays out

KEEP_OUT :

.env .git .venv .quark

Secrets and history never enter the box.

Why copy

Mounted: the agent changes your real files as it goes.

Copied: your files are **untouched**.

Where the copy lives

`/work`, a **tmpfs**, 64 MB.

Memory-backed. Gone when the box goes.

Even less power

User **65534**.

Nobody, in unix terms.

Capped output

`run_in_box` writes output to `/tmp/out` .

Then reads `head -c MAX_OUT` .

A huge output can't flood the context.

The LIMITS list

The flags live in one list.

Easy to read. Easy to change.

Discard the environment

Whatever happened in the box can be thrown away.

That's the point.

review()

At the end, it lists files **created or changed**.

It compares against a marker made at the start.

Not deleted

Deleted files aren't listed.

That's a limit of the approach.

Copy out what you want

It asks.

Say yes: files copied to `./sandbox-out`.

Everything else is discarded.

The person decides what escapes

Nothing leaves the box unless you say so.

Where commands run is output.

Asking you first is a guardrail.

Seeing limits bite

Sed'd copies: `TIMEOUT` of 5, `MAX_OUT` of 300.

The README shows each hit its limit.

What sandboxing can be

- **where:** container, VM, remote machine
- **what it reaches:** files, network, time, memory
- **how files get in:** mounted or copied
- **what comes out:** nothing, or only what you approve

What's still open

- `kill -9` on the harness leaves the container behind
- the harness itself is **not** sandboxed
- `--network none` means **no installs**

What to take away

Rule: sandboxing is output choosing where tools run, with the kernel enforcing what they can reach.

Notice what sandboxing never does

It never changes what the model sees.

It never decides when the loop stops.

It never decides whether to ask.

What's missing

The agent is watched, guarded, and boxed.

But the world is unreliable.

The API may fail. A command may hang. quark may crash.

Next: Resilience

Lesson 8 keeps going when things fail.